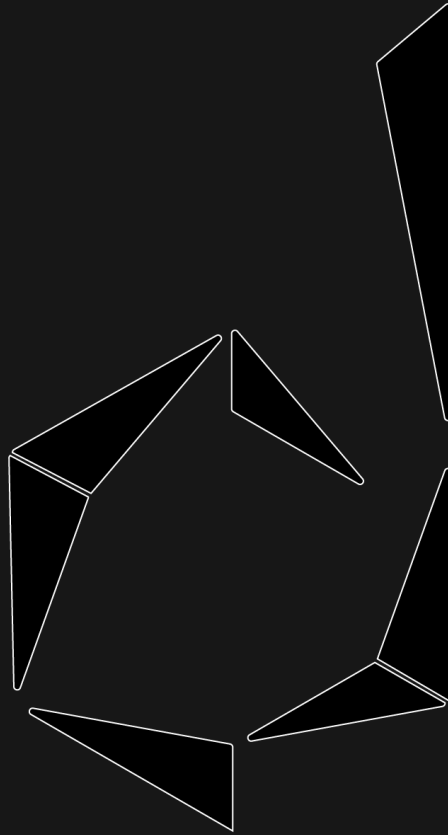


INGENIERÍA MECATRÓNICA



DI_CERO

DIEGO CERVANTES RODRÍGUEZ

DEVOPS Y CONTROL DE VERSIONES DE ARCHIVOS DE TEXTO PLANO

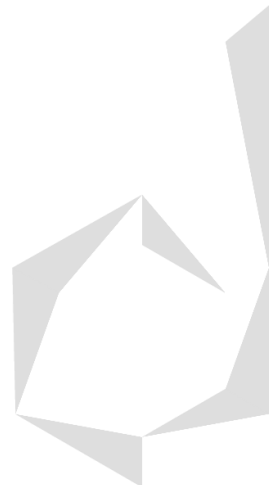
GIT Y GITHUB

GitHub: Buenas Prácticas
Pull Requests (Merge de Ramas)

Manual de Git

Contenido

Buenas Prácticas en Pull Requests (PR)	2
1. git branch nombre_rama_nueva	2
a. git branch -D nombre_rama_a_borrar	2
2. git checkout nombre_rama.....	2
3. git branch	2
4. git add nombre_archivo.extension	3
Referencias.....	5

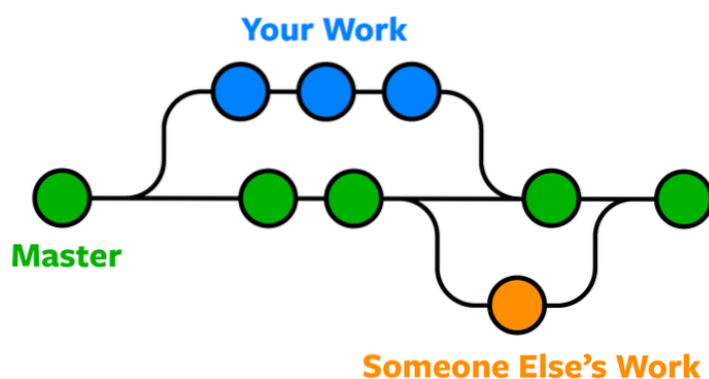


Buenas Prácticas en Pull Requests (PR)

Una **Pull Request** se refiere a la acción de crear una **rama** a través de otra para hacerle un **cambio** a la original, como un **feature** (nueva funcionalidad), arreglar un **bug**, etc. y que otro programador la pueda checar para verificar que está bien hecha la edición del código, por lo que para crear este tipo de elementos primero que nada se debe elegir la **branch correcta**. Por buenas prácticas, se considera que hay una rama principal llamada **main** y una copia suya de pruebas llamada **staging_develop** que siempre debe estar actualizada para que sean iguales. A través de la rama de **desarrollo develop** se crean las ramas de **feature** para agregar nuevas funcionalidades o mejoras del código y las ramas **hotfix** para arreglar errores.

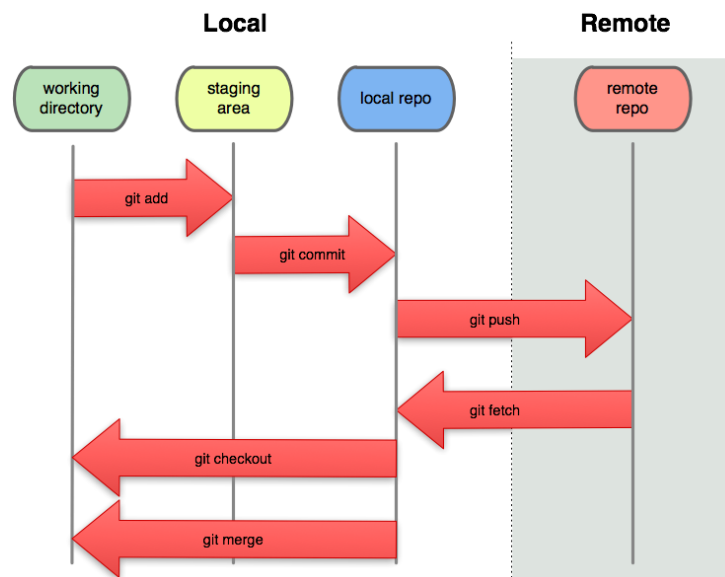
*Es importante mencionar que el apuntador del commit más reciente de la **rama** donde nos encontremos actualmente tiene un nombre especial para diferenciarse de los demás y ese nombre es **HEAD** o **cabecera**.*

1. **git branch nombre_rama_nueva**: Comando que sirve para crear una rama nueva, moviendo así el **HEAD** hacia ella para que ahora se pueda crear una nueva línea de tiempo. Recordemos que con el comando **git show** podemos ver las ramas a las cuales está apuntando el **HEAD**. La primera rama que se indique dentro del paréntesis es en la que nos encontramos.
 - a. **git branch -D nombre_rama_a_borrar**: Comando para borrar una rama, cuando queramos deshacernos de una rama debemos estar situados sobre una distinta a la que queremos eliminar.
2. **git checkout nombre_rama**: Comando que sirve para moverme de una rama a otra. Esto se debe hacer porque cuando se crea una nueva rama no nos movemos automáticamente a ella, solamente se crea una copia del commit actual, esa es la razón por la cual cuando se crea una rama nueva, el **HEAD** apunta a dos ramas a la vez, porque en ese nodo o commit conviven dos ramas. Una vez que nos hayamos movido de rama, para checar que en efecto nos hemos cambiado podemos usar el comando **git status** o **git branch**.
 - a. El **HEAD** dejará de estar en dos ramas a la vez cuando realicemos nuestro primer **commit** en la rama nueva, ya que en este punto se creará una nueva línea de tiempo en la rama recién creada. (que puede tener cualquier nombre como “cabecera” o “nueva_rama”).
3. **git branch**: Comando que muestra un listado de todas las ramas existentes en el proyecto, resaltando con el **color verde** y un asterisco * la rama en la cual nos encontramos actualmente.



Lo primero que se debe considerar para construir una buena **Pull Request (PR)** es **fijar un propósito en específico de lo que se quiera cambiar del código, ya sea un **feature** o un **bug****, para ello se consideran dos cosas, una es la descripción del **PR** para que la otra persona entienda el objetivo del cambio y en cuestión técnica debemos tener cuidado de afectar el menor número de archivos posibles, para que cuando las demás personas lo revisen, no ocupen mucho de su tiempo al hacerlo, por lo que no se debe usar el comando **git add -A** al realizar commits, sino más bien, **añadir a la rama solamente los archivos importantes que deben ser checados**, para ello, se ejecuta de la siguiente forma el comando:

4. **git add nombre_archivo.extension**: Comando que asocia al **Staging Area** un archivo en específico de la carpeta.
 - a. Es importante realizar esto desde un inicio cada vez que mandamos commits porque una vez que el **Pull Request** haya sido creado para hacer **merge** entre la rama de **feature** o **bug** con la **rama de desarrollo develop**, no se pueden realizar cambios en el número de archivos que han sido afectados en la rama del **PR**.
 - b. En una **PR**, la casilla ☒ **Viewed** junto a cada archivo en la pestaña **Files Changed** es una herramienta personal que permite a cada usuario marcar los archivos que ya ha revisado, ayudando a organizar su propio progreso durante la revisión. Esta casilla no es visible para otros revisores, no afecta el estado del **Pull Request** ni cuenta como aprobación, y no sirve para indicar a los demás que no revisen un archivo. Además, si el archivo se actualiza con nuevos cambios, GitHub desmarcará automáticamente la casilla para que el usuario sepa que debe revisarlo nuevamente.
 - c. Además, dentro de la pestaña **Files Changed** se puede añadir comentarios para indicar al revisor un comentario con detalles de lo que se estaba realizando en cada línea de código.



5. Para realizar una buena descripción del propósito de una **PR**, una vez que nos hayamos asegurado de afectar el menor número posible de archivos del repositorio se debe añadir lo siguiente:
 - a. Un buen título y descripción del cambio:

- i. Al inicio de la descripción de un **Pull Request** lo que se hace es proporcionar algo llamado **TL;DR**, que significa **Too Long; Didn't Read** y se refiere a una breve descripción antes de poner tres guiones de separación (---) y luego entrar de lleno a lo que se realizó en el **PR**.
- ii. En la descripción del PR se puede dar hasta contexto e instrucciones de como realizar pruebas al **feature** o arreglo del **bug** a través del siguiente formato:

¿Porque hacemos esto? requested () to do ()

Pasos de testeo (como correr pruebas del cambio de forma local):

1. **Dependencias necesarias, variables de entorno a añadir, etc.**
- iii. Dentro de la descripción, se puede hacer referencia a tickets creados en la plataforma de DevOps que estemos utilizando, ya sea Trello, Azure DevOps, etc. o a issues creados en el mismo repositorio, para ello se utiliza el formato de **#** seguido del número del issue o ticket: **#Issue_o_Ticket_DevOps**
- iv. Por último, se pueden agregar imágenes, tablas, ecuaciones matemáticas, gráficas o videos utilizando el formato *markdown* para explicar los pasos de testeo o conceptos del **PR**. Para ello se cuenta con la siguiente documentación de GitHub:

<https://docs.github.com/es/get-started/writing-on-github/working-with-advanced-formatting>

- v. Si en el **Pull Request** tenemos varios archivos en la pestaña de *Files Changed*, lo que podemos añadir a la descripción es un enlace a un commit en específico de toda la rama que se refiera en específico a los cambios que quiero que sean verificados, para ello debemos ver el número que tiene asociado cada commit llamado hash y crear un enlace con lenguaje markdown de la siguiente manera:

hash_number [**e1a2b3c**](<https://github.com/commit/e1a2b3c>)



6. Si hay pruebas automatizadas en el repositorio, al publicar nuestra **Pull Request** estas se realizarán sobre nuestros cambios y si estas pruebas fallan, al dar clic en el botón de Details, podremos ver dónde y porqué.

Referencias

Rishav Chanda, "Next Level GitHub Profile README (NEW 2022) (Markdown template) | Improve Your Github profiles", 2022 [Online], Available: <https://www.youtube.com/watch?v=G-EGDH50hGE&t=1s>

